

For high-level DSA, people don't use *everything* in STL. They use a **small, brutal toolkit** that maps directly to problem patterns. The rest exists, but these few win wars.

Let's go container by container, with *why they dominate, what advantage they have over others, and how to recognize them at a glance*.

vector

This is the backbone of DSA.

Why it's used:

- Contiguous memory → cache-friendly → fastest iteration
- $O(1)$ random access
- Works perfectly with sorting, binary search, two pointers, prefix sums

Advantage over others:

- Faster than list/deque for most tasks
- Simpler than arrays, safer than pointers

When your brain should scream "vector":

- You see indexing
- You see sorting
- You see ranges, subarrays, prefix/suffix
- Input size known or manageable

Instant recognition trick:

If the problem smells like **array math**, the answer starts with `vector`.

string

This is just a specialized `vector<char>` with bonuses.

Why it's used:

- Handles dynamic resizing
- Rich built-in operations
- Plays well with hashing, sliding window, DP

Advantage:

- Faster and safer than manual char arrays
- Direct compatibility with STL algorithms

Use it when:

- You see substrings

- Palindromes
- Pattern matching
- Sliding window on characters

Instant recognition:

If characters matter and order matters → `string`.

`unordered_map`

This is the king of lookup problems.

Why it's used:

- Average $O(1)$ insert/find
- Perfect for frequency counting
- Turns nested loops into single passes

Advantage over `map` :

- Much faster
- No ordering overhead

When to use:

- Frequency count
- "Have you seen this before?"
- Two-sum, subarray sum, prefix logic
- Mapping values to indices

Instant recognition:

If the problem says **count, frequency, existence, seen before**, reach for `unordered_map`.

`map`

Ordered structure with guarantees.

Why it's used:

- Keys stay sorted
- $O(\log n)$ guaranteed
- Useful when order matters

Advantage over `unordered_map` :

- Sorted traversal
- No hash collision disasters
- Works for range queries

Use it when:

- You need smallest/largest key repeatedly
- Order of keys matters
- Range operations

Instant recognition:

If the problem mentions **minimum, maximum, next, previous, ordered**, use `map`.

`unordered_set`

Presence detector.

Why it's used:

- $O(1)$ lookup
- No duplicates
- Minimal memory overhead compared to map

Advantage:

- Cleaner than map when only existence matters

Use it when:

- Uniqueness
- Cycle detection
- Fast membership checks

Instant recognition:

If you only care **whether something exists**, not how many times → `unordered_set`.

`stack`

This is structure, not storage.

Why it's used:

- Enforces LIFO
- Elegant for nested logic

Advantage:

- Expresses intent better than vector

Use it when:

- Matching parentheses
- Undo behavior
- Previous/next greater element

- DFS (iterative)

Instant recognition:

If the problem has **nesting, reversal, backtracking, previous state**, it's a stack problem.

queue

Controlled processing order.

Why it's used:

- FIFO behavior
- Natural for level-based problems

Advantage:

- Perfect for BFS

Use it when:

- Shortest path (unweighted)
- Level traversal
- Simulation

Instant recognition:

If the problem moves **level by level**, think queue .

deque

The silent assassin.

Why it's used:

- Push/pop from both ends in $O(1)$

Advantage:

- Combines vector + queue traits

Use it when:

- Sliding window maximum/minimum
- Window that expands and contracts from both ends

Instant recognition:

If you see **sliding window + max/min**, deque is almost guaranteed.

priority_queue

Greedy problems live here.

Why it's used:

- Fast access to min/max element
- $O(\log n)$ operations

Advantage:

- Outperforms sorting repeatedly
- Perfect for “best so far” logic

Use it when:

- K largest / smallest
- Scheduling
- Merging sorted lists
- Dijkstra’s algorithm

Instant recognition:

If the problem asks **best, top, minimum, maximum repeatedly**, this is it.

`pair` and `tuple`

Data bundlers.

Why they’re used:

- Compact representation
- STL compatibility

Advantage:

- Cleaner than structs for small groupings

Use them when:

- Coordinate problems
- Storing value + index
- Graph edges

One-glance identification cheat rules

- Counting → `unordered_map`
- Presence only → `unordered_set`
- Order matters → `map`
- Fast index access → `vector`
- Sliding window max/min → `deque`
- Best choice repeatedly → `priority_queue`
- Nesting or rollback → `stack`
- Level order / BFS → `queue`

Advanced pro habits (this separates serious DSA people)

- Default to `vector`
- Use `const &` everywhere
- Avoid `map` unless order is required
- Prefer `unordered_*` but switch to ordered if TLE or collisions
- Replace recursion + stack overflow with `stack` when needed
- Combine containers (map of vectors, vector of pairs)

Final truth (this matters)

You don't "choose" a container.

The **problem leaks the container choice** through its constraints.

Once your eyes catch:

- access pattern
- ordering requirement
- repetition
- direction of flow

...the STL choice becomes obvious in under 5 seconds.

Master these few, and 99% of DSA problems stop feeling random.